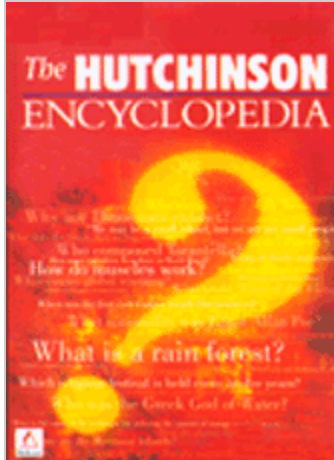


What are the main stages involved in the development of large commercial information and communication systems?



Source: The Hutchinson Unabridged Encyclopedia with Atlas and Weather Guide.

Publisher Information: Helicon, 2018.

Publication Year: 2018

The software development life cycle is a widely-used process used to develop software. Learn more about the SDLC phases and models.

The use of software development life cycle (SDLC) frameworks has become mostly standard in software development. Most companies will use an SDLC to guide development, though not every company will use the same methodology. Whether you develop software yourself or plan to work with a dev team in the near future, you should expect to encounter some kind of SDLC framework.

Familiarity with popular frameworks like Agile and Scrum—as well as the unique vocabulary and concepts these frameworks use—may be necessary to effectively communicate with some developers and dev teams.

Table of Contents

What is the SDLC?

The software development life cycle is a set of steps that enables the process of developing new software. In practice, most SDLCs offer a mix between a software development methodology and a description of the development process.

These SDLCs attempt to describe how software development happens and prescribe specific phases and strategies that developers can use to create large software projects. The framework covers the entire cycle of development, including all the tasks needed to plan, build, test, and deploy the software.

Using an SDLC can help developers better understand the software development process and guide the team through developing new software.

What are the phases of the SDLC?

The specific phases of the SDLC can vary depending on the particular framework a business or development team is using. Most SDLC models, however, include these phases:

- **Requirement analysis:** Determining user requirements for the software.
- **Design:** Planning how they'll create software that meets those requirements.
- **Development:** Building the functional product based on user requirements and planning from the design phase.
- **Testing:** Testing the functionality of the product.
- **Deployment:** Making the product available to end-users.
- **Maintenance:** Updating the product, discussing the product with users, and ensuring the product functions correctly.

Some models of the SDLC may also describe an ideation phase that comes before the requirement analysis phase. This early ideation provides the development team with ideas and concepts they can discuss with clients, potential users, and other stakeholders whose needs will determine the software's requirements.

In practice, the divisions between these phases may not always be perfectly clean. Depending on the development team's structure and strategy, some development phases may overlap. Other phases may recur throughout the development cycle.

Requirement analysis, for example, may occur throughout a project's development. Client needs can evolve over time, and early miscommunications may require adjustments later in the development process.

Testing, designing, and development may all occur again after deployment as part of the maintenance and update development process for the software that a team continues to support after launch.

Different SDLC models

Not everyone models the software development life cycle in the same way. Different types of SDLC models are best suited for different teams and software development projects. The most common SDLC models include Waterfall, Scrum, Agile,

Waterfall model

Most descriptions of the Waterfall model outline five distinct phases:

- Requirements determination
- Design
- Implementation

- Verification
- Maintenance

Some variations on the model break the design phase down further into logical design and physical design phases. Others may include a deployment phase between the verification and maintenance steps. With the Waterfall approach, each main phase of software development occurs one after the other until the product is complete and available to end-users. Completion of one phase is typically required before the next can begin. Each new phase also requires deliverables from the previous phase, so progress flows primarily in one direction. As a result, these phases "waterfall" into the next.

Limitations

The Waterfall approach is not iterative and is generally inflexible compared to alternatives. If developers strictly follow this approach, they won't have much room to prototype, experiment with potential solutions, or revise work without significantly disrupting the development process.

If a client needs change during the development process—and the project's requirements change as a result—the Waterfall approach may prevent a development team from adapting effectively. Redesign, redevelopment, and retesting may all be necessary, leading to slower and more expensive development. Despite the limitations of the Waterfall approach, it remains popular, though not nearly as popular as it was before the 2000s.

Read more on *Project-Management.com*: Agile vs. Waterfall: Differences in Software Development Methodologies

Prototyping model

As the name suggests, the prototyping model involves a prototyping phase, where the development team experiments with potential software features and design strategies.

Customers, clients, or potential users then test the prototypes and provide feedback. The development team integrates this feedback into their design and requirements planning phases, then develops new prototypes. When the customer is satisfied with the prototype, development proceeds normally.

This model improves on many of the weaknesses of the Waterfall model. It's one of the most popular SDLCs, balancing the simplicity of the Waterfall model with some improvements that add complexity to the software development process but also help mitigate risk.

Iterative approach

With the iterative approach, the development team doesn't assume the first version of their software is the one they will deploy. Instead, the team prepares for multiple cycles of development—or software iterations—before they will arrive at a finished project that's ready for deployment.

Many iterative approaches are also incremental, meaning the overall project scope is broken down into smaller chunks that the team can tackle one at a time. These smaller project chunks, once complete, can be individually tested or delivered to clients for feedback, allowing the development team to learn more about what the client wants from the final project.

The iterative approach is similar to the prototyping model, but revisions to project requirements continue throughout the development process. Instead of having a dedicated prototyping phase where requirements may change, the team expects that project requirements and scope may remain somewhat unstable until the project is complete and deployed.

Like the prototyping model, the iterative approach builds on the Waterfall model and provides an answer to many of its shortcomings. Unlike the prototyping approach, however, clients are closely involved in much of the development process, rather than just the initial prototyping phase.

Agile

Agile is an umbrella term for methodologies based on the Agile Manifesto for software development. The manifesto outlines 12 principles for software development, including welcoming changing requirements, frequently delivering working software, and communicating via face-to-face conversation.

This approach is suited for projects where requirements are frequently changing and the development team needs to adapt quickly as development proceeds—hence the name "Agile."

Most Agile approaches to software development leverage a few of the same management and development practices like pair programming, stand-ups, and sprints. These practices help developers align their work with Agile principles.

In the ideal Agile workplace, teams are self-organizing and collaboration-oriented. They frequently deliver working software and measure progress by the amount of working software they produce.

Agile is especially popular in startup environments, where rapid development of working software can help a company secure funding and build the business's reputation or client base. Like other SDLCs, however, Agile development does have drawbacks.

Critics of Agile say the fast pace of Agile development can limit the potential of new projects and encourage developers to play it safe, focusing on developing working software with existing competencies rather than experimenting, learning, and acquiring new skills.

Related: What is an Agile Coach?

Scrum

Scrum is a derivative of the Agile framework that provides a deliverable-oriented approach to product development and the SDLC.

In Scrum project management, the project team consists of a project manager, product owner, Scrum Master, and various cross-functional team members. The product owner is responsible for maximizing the value of the product, while the Scrum Master keeps the team aligned with the Scrum framework.

This framework relies heavily on "sprints" – short periods of time, usually around two to four weeks in length, during which team members work on a small part of the project's full scope. At the end of the sprint, this work is delivered to the client.

As with all other Agile frameworks, Scrum focuses heavily on working software deliverables as a metric of progress. Most of the criticisms of Agile in general also apply to Scrum.

Related: Tips for Implementing Scrum Best Practices

Kanban

Kanban is another popular framework used to implement Agile software development. Unlike other Agile methodologies, Kanban makes small changes to the typical software development team structure, making it a good fit for more traditional organizations. Whereas Scrum focuses on delivering working software within a set amount of time, Kanban focuses on breaking down work into smaller tasks, visualizing them, and organizing them.

The methodology revolves around a project management tool called the Kanban board, which helps the team visualize work. This board indicates the tasks currently being performed, completed tasks, and future tasks.

Many teams will use digital Kanban board tools to organize their projects, but physical boards can also be useful, though tracking large numbers of tasks with these boards may be labor-intensive.

The methodology offers no tools to help the development team manage the project's timeline, however, meaning estimating how long a project will take to complete can be difficult. Complex projects may also result in highly complex Kanban boards that are difficult for the team to understand.

Read more on TechRepublic: Scrum or kanban: Which agile software development tool is best for your project?

Pros and cons of using an SDLC model

A formal framework for software development helps guide developers through the process of creating new software. For most projects, using an SDLC model can help clarify project goals, streamline management, and reduce the risk of budget overages or missed deadlines.

There can be disadvantages to adopting an SDLC model, however. The wrong framework, for example, can limit team flexibility or create unrealistic expectations about how development will proceed. A framework that is too flexible or loose can also create problems, especially for larger teams where team scale can make management, organization, and communication much more difficult.

Streamlining software development with an SDLC

With a software development life cycle, a development team can describe its process for creating new software. This description can help the developers both better understand the project and create frameworks for guiding software development.

The original SDLC was the Waterfall model, a straightforward approach with significant limitations. For larger software projects, most businesses will probably use a different approach—like the prototyping model, the iterative approach, or one of the many variations of the Agile methodology.

Each of these distinct SDLC models comes with its own advantages and disadvantages. In the right situation, most of these methodologies can help guide a software development team through the development process. However, not every SDLC approach will be appropriate for every project.

Read next: What are the Key Risks of Software Development Outsourcing?

~~~~~

By Devin Partida

Reported by Author